

## CSCE 605 COMPILER DESIGN · MODULE 05 · LECTURE NOTES

# Names, Scopes, and Symbol Tables

*Connecting every use to its definition.*

## MODULE OUTCOMES

- Implement lexical scoping with a scope stack.
- Handle shadowing, forward references, and recursion.
- Explain the difference between declaration, definition, and use.
- Implement overload resolution and namespacing.
- Produce useful name-resolution diagnostics.

## 1 Lexical scoping

### LEXICAL SCOPING IS DETERMINED BY THE TEXT

**A name refers to the nearest enclosing declaration in the source text**, and because the source text is available at compile time, the resolution can be computed once and recorded.

**Dynamic scoping instead resolves to the nearest declaration on the call stack at runtime**, which means a function's meaning depends on who called it — the same function body can refer to different variables on different calls.

**Dynamic scoping lost decisively, and for good reasons.** It prevents local reasoning (you cannot understand a function without knowing its callers), defeats most optimisation (the compiler cannot know what a name refers to), and makes refactoring unsafe (renaming a local in one function can change another's behaviour).

**It survives in narrow places where caller-dependence is exactly the point:** shell environment variables, Emacs Lisp's dynamic defaults, exception handler lookup, and thread-local or task-local context. **Each of those is a case where you genuinely want the caller to be able to influence the callee**, which is worth noticing — the feature is not bad, it is just a bad default.

```

struct Scope { unordered_map<Symbol, Decl*> names; Scope* parent; };

Decl* lookup(Scope* s, Symbol name) {
    for (; s; s = s->parent)
        if (auto it = s->names.find(name); it != s->names.end())
            return it->second;
    return nullptr;          // undefined -- report it
}

void declare(Scope* s, Symbol name, Decl* d) {
    auto [it, inserted] = s->names.emplace(name, d);
    if (!inserted)
        error(d->span, "redefinition of '%s'", name)
            .note(it->second->span, "previous definition here");
}

// Shadowing an OUTER scope: legal. Redefining in the SAME scope:
// an error. The distinction is just which map held the name.

```

**Note the two spans in the diagnostic.** 'Redefinition of x' tells the user something they can already see; 'previous definition here' tells them the thing they could not find, and it is the difference between a complaint and an explanation (Module 03 §4).

## 2 What makes it harder

| Case                   | The problem                                                                                                   | The handling                                                                                                                                                                                          |
|------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Forward references     | A top-level function is called before it is <b>declared</b> , which is normal and expected in most languages. | <b>Two passes</b> — see the callout below.                                                                                                                                                            |
| Mutual recursion       | Each of two functions refers to the other, so neither can be resolved first.                                  | <b>Falls out of the two-pass structure for free</b> , which is part of why that structure is universal.                                                                                               |
| Closures               | A <b>captured variable can outlive the stack frame it was declared in</b> .                                   | The resolver must mark which locals are captured and how, so that code generation can box them (§3).                                                                                                  |
| Overloading            | One name has several definitions, distinguished by argument types.                                            | Name resolution produces a candidate <i>set</i> , and type checking selects from it (Module 06). The two phases must be interleaved.                                                                  |
| Imports and namespaces | <b>Scope is no longer purely lexical</b> — names enter from elsewhere, possibly cyclically.                   | An explicit import-resolution pass before name resolution, with cycle detection.                                                                                                                      |
| Generics and templates | <b>What a name means can depend on the instantiation</b> .                                                    | <b>Two-phase lookup</b> : resolve what you can at definition, defer dependent names to instantiation. <b>This is where C++ becomes genuinely hard</b> , and it is the source of the typename keyword. |

**TWO PASSES, AND WHY BOTH ARE NEEDED**

**Pass 1 collects declarations** into their scopes without resolving anything. When it finishes, every scope knows every name declared directly within it, though no use has been connected to anything.

**Pass 2 resolves uses** against the now-complete scopes, recording for each use a pointer to the declaration it refers to.

**Forward references work** because the declaration was collected in pass 1, before any use was examined in pass 2. **And mutual recursion works for free** — neither function needs the other to have been processed first, because neither is resolved until both are collected.

**Inside a function body, though, scoping is usually sequential:** a local variable used above its declaration is an error, even though a top-level function used above its declaration is fine. **Those are deliberately different rules** — sequential scoping inside a body is what makes a reader able to follow the code top to bottom, while top-level order-independence is what makes a file's layout a matter of style. Implement them as two different behaviours, not one with exceptions.

### 3 Closures and capture

#### CAPTURE IS THE HARD PART OF SCOPING

A nested function that references a local of an enclosing function *captures it*, and the resulting closure may outlive the stack frame that local was allocated in — which means the variable cannot simply live on the stack.

**Capture by value copies the variable at closure-creation time.** Simple, safe, and later mutations by either side are invisible to the other. This is C++'s [=] and the default in several languages.

**Capture by reference shares the variable**, so mutations are mutually visible — which means **the variable must be heap-allocated (boxed) as soon as anything captures it by reference**, and the resolver is what determines that, long before code generation.

**The classic bug is capturing a loop variable by reference:** every closure created in the loop shares one variable, so after the loop they all observe the final value, and a loop that created ten callbacks produces ten identical ones. **JavaScript introduced `let` with per-iteration binding specifically to fix this**, and C++ requires you to state the capture mode explicitly rather than guessing — two different language-level answers to the same resolver-level problem.

- **For each name use, the declaration it resolves to.** A pointer, computed once, consulted by type checking, IR generation, and every later phase. **Nothing should ever re-resolve a name.**
- **For each local, whether it is captured and how.** This decides stack versus heap allocation, and it must be known before code generation lays out the frame.
- **For each function, its captured set** — which becomes the layout of the closure environment object.
- **For each scope, its depth and its frame-slot assignment**, if you resolve names to slots rather than to strings.
- **Resolving to slot indices rather than to names is the single largest interpreter speedup available.** A variable access becomes an array index instead of a hash table lookup, and the hash lookup was happening on every access in the inner loop. Crafting Interpreters measures this and the difference is substantial.

### 4 Diagnostics this phase owes

**'UNDEFINED NAME' SHOULD SUGGEST**

**The overwhelming majority of undefined-name errors are typos**, so compute the edit distance from the unknown name to every name in scope and suggest the closest one.

**Suggest only within a small distance** — one or two edits on a name of reasonable length — **and suggest only one candidate**. A list of five possibilities is noise that the user has to filter, and a confident single suggestion that is right 90% of the time is far more valuable.

**Also check scopes the name is *not* in**. 'There is no local named count, but there is a field this.count' or 'did you mean the module-level count?' is frequently exactly the right answer, and it costs one extra lookup.

**And report the original declaration site on a shadowing warning** — two spans, as in §1. **The second span is what turns a complaint into an explanation**, and it is the single most reusable idea in compiler diagnostics.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                       | What it covers for this module                                                                                                      |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Nystrom — Crafting Interpreters, 'Resolving and Binding' and 'Closures' (free) | <b>The two-pass resolver and closure capture, implemented and measured.</b> The slot-index optimisation of §3 is here with numbers. |
| Stanford CS143 — semantic analysis lectures (free)                             | Symbol tables, scoping, and the structure of §2.                                                                                    |
| Cooper & Torczon — Engineering a Compiler, chapter 5                           | Symbol table implementation choices and their costs.                                                                                |
| rustc dev guide — name resolution (free)                                       | A production resolver handling modules, imports, macros, and cycles — the §2 complications at full scale.                           |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a scope stack with enter, leave, declare, and lookup.
- 2 **Produce a redefinition error with two spans**, and a shadowing warning with two spans.
- 3 Implement the two-pass structure and verify forward references and mutual recursion both work.
- 4 **Verify that sequential scoping inside a body still rejects** a local used before declaration.
- 5 Implement closures with capture by value, then by reference.

- 6 **Reproduce the loop-variable capture bug** and then fix it with per-iteration binding.
- 7 Determine which locals are captured and box only those. Measure the allocation count before and after.
- 8 **Resolve names to slot indices** and measure your interpreter before and after.
- 9 Implement edit-distance suggestions for undefined names and test them on real typos.
- 10 Add a check for 'there is a field with this name' and report it.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Contrast lexical and dynamic scoping, and say why one won.
- 2 Where does dynamic scoping still make sense?
- 3 How does the scope stack distinguish shadowing from redefinition?
- 4 Why are two passes needed, and what do they solve together?
- 5 Why is scoping inside a body sequential when top level is not?
- 6 Contrast capture by value and by reference, and say what boxing is for.
- 7 Describe the loop-variable capture bug and two language-level fixes.
- 8 What should the resolver record for later phases?
- 9 What makes a good undefined-name diagnostic?